

Prometheus - Cheat Sheet

Prometheus is a time-series database designed for monitoring. It pulls metrics from your services on a schedule, stores them with a label-based data model, and lets you query them with PromQL. Born at SoundCloud in 2012, donated to the CNCF in 2016, second project to graduate after Kubernetes.

The pull model (Prometheus calls you) is the design choice that defines everything else. Service discovery, scrape configs, exporters all flow from “Prometheus needs to find what to scrape”.

Architecture

Several pieces, often deployed together.

Component	Role
Prometheus server	Scrapes targets, stores time series, runs queries
Exporter	Translates an external system's metrics into Prometheus format (<code>node_exporter</code> for OS metrics, <code>mysqld_exporter</code> for MySQL, etc.)
Pushgateway	For short-lived jobs that can't be scraped. Acts as an intermediary.
Alertmanager	Receives alerts from Prometheus, deduplicates, routes, and notifies
Client libraries	Embedded in your services. Expose <code>/metrics</code> .
Service discovery	Tells Prometheus where to scrape (Kubernetes, Consul, file, DNS, etc.)

The cycle: services expose `/metrics` over HTTP, Prometheus scrapes them every N seconds, stores samples, evaluates recording and alerting rules.

Data model

Every sample is `(metric_name, labels, value, timestamp)` . The combination of metric name and labels identifies a unique time series.

```
http_requests_total{method="GET", status="200", endpoint="/users"} 1027 @ 1716000000
```

- `http_requests_total` is the metric name.

- `method` , `status` , `endpoint` are labels (key-value pairs).
- `1027` is the sample value.
- `1716000000` is the unix timestamp.

The label set is what gives Prometheus its slicing power and what makes cardinality dangerous (more on that below).

Metric names follow `lowercase_with_underscores` . Counters end in `_total` . Durations end in `_seconds` . Sizes end in `_bytes` . These conventions matter. Tooling depends on them.

Metric types

Four types. The client library you use in your app picks the right primitive.

Counter

Monotonically increases. Resets on restart.

```
http_requests_total{status="200"} 1027
```

Use for requests served, errors raised, bytes processed. Anything you only add to.

You don't query the counter directly. Apply `rate()` or `increase()` to see how fast it's growing.

Gauge

Goes up and down.

```
queue_depth 42
db_connections_active 12
memory_bytes_used 8_345_123_456
```

Use for temperature readings, current count of in-flight things, free memory.

Histogram

Samples observations and counts them in buckets. Pre-computed at scrape time.

```
http_request_duration_seconds_bucket{le="0.1"}    89
http_request_duration_seconds_bucket{le="0.5"}    135
http_request_duration_seconds_bucket{le="1.0"}    147
http_request_duration_seconds_bucket{le="+Inf"}    150
http_request_duration_seconds_sum                  42.7
http_request_duration_seconds_count                150
```

Use for request latencies, response sizes. Anything where you want percentiles.

Query histograms via `histogram_quantile` :

```
histogram_quantile(0.95,  
    rate(http_request_duration_seconds_bucket[5m]))
```

Summary

Similar to histogram but quantiles are pre-computed in the client. Less flexible (can't aggregate across instances) but more accurate per-instance.

```
http_request_duration_seconds{quantile="0.5"}    0.12  
http_request_duration_seconds{quantile="0.9"}    0.34  
http_request_duration_seconds{quantile="0.99"}   1.21
```

Modern advice: use histograms. They aggregate across replicas. The native histograms feature in Prometheus 2.40+ makes them even better.

Exposition format

The text format Prometheus scrapes. Plain HTTP `GET /metrics` :

```
# HELP http_requests_total Total HTTP requests  
# TYPE http_requests_total counter  
http_requests_total{method="GET",status="200"} 1027  
http_requests_total{method="POST",status="201"} 35  
http_requests_total{method="GET",status="500"} 2  
  
# HELP queue_depth Current queue depth  
# TYPE queue_depth gauge  
queue_depth{queue="orders"} 12  
queue_depth{queue="emails"} 0
```

`# HELP` and `# TYPE` lines are metadata. The rest are samples.

OpenMetrics is the standardized version of this format. Mostly identical.

PromQL essentials

The query language. Four expression types.

Instant vector

Latest value for each matching time series, at the evaluation time.

```
http_requests_total
http_requests_total{status="200"}
http_requests_total{status=~"5.."}    # regex match: 5xx
```

Range vector

Samples over a time range. Used by rate functions.

```
http_requests_total[5m]
```

Returns a list of samples from the last 5 minutes per time series. You don't display this directly. Feed it to `rate()` or `increase()`.

Scalar

A single number.

```
42
sum(http_requests_total)
```

String

Rarely used.

Common functions

```
# Per-second rate of a counter over 5m
rate(http_requests_total[5m])

# Total increase of a counter over a window
increase(http_requests_total[1h])

# Instant rate (uses only last 2 samples). Twitchy. Use sparingly.
irate(http_requests_total[1m])

# Average of a range
avg_over_time(queue_depth[10m])

# Max, min, sum, count over a range
max_over_time(cpu_usage[1h])
min_over_time(cpu_usage[1h])

# Difference between first and last
delta(temperature[10m])

# Quantile from a histogram
histogram_quantile(0.99, rate(http_request_duration_seconds_bucket[5m]))
```

`rate` divides by the time window, so the result is “per second”. `increase` is `rate * duration` and gives “total over the window”. Both work only on counters.

Aggregation operators

Combine many time series into fewer.

```
# Sum across all matching series
sum(rate(http_requests_total[5m]))

# Group by a label
sum by (status) (rate(http_requests_total[5m]))

# Exclude a label
sum without (instance) (rate(http_requests_total[5m]))

# Other aggregators
avg, min, max, count, stddev, stdvar, topk, bottomk, quantile
```

`topk(5, ...)` gives the 5 highest. `quantile(0.95, ...)` is a different beast from `histogram_quantile`. This one aggregates pre-existing values. `histogram_quantile` derives percentiles from a histogram.

Operators

Math between vectors with matching labels:

```
# Error rate as a fraction
sum(rate(http_requests_total{status=~"5.."}[5m]))
/
sum(rate(http_requests_total[5m]))

# CPU saturation check
node_cpu_usage > 0.8

# Boolean comparison (1 if true)
node_cpu_usage > bool 0.8
```

Vector-to-vector matching requires labels to align. Use `on (label)` and `group_left` / `group_right` for many-to-one joins:

```
deployment_replicas_running
/ on (deployment)
deployment_replicas_desired
```

Scraping configuration

prometheus.yml:

```
global:
  scrape_interval: 15s
  evaluation_interval: 15s

scrape_configs:
  - job_name: "java-app"
    metrics_path: /actuator/prometheus
    static_configs:
      - targets: ["app1:8080", "app2:8080"]

  - job_name: "node"
    static_configs:
      - targets: ["host1:9100", "host2:9100"]

  - job_name: "kubernetes-pods"
    kubernetes_sd_configs:
      - role: pod
    relabel_configs:
      - source_labels: [__meta_kubernetes_pod_annotation_prometheus_io_scrape]
        action: keep
        regex: true
```

`scrape_interval` defaults to 15-30s in most setups. Tighter intervals mean more storage and CPU.

`relabel_configs` rewrites labels before storage. The pattern above keeps only pods annotated with `prometheus.io/scrape: "true"`.

Service discovery

Prometheus needs to know what to scrape. Static lists work for fixed targets. Production usually uses discovery.

SD method	Source
static_configs	Hardcoded targets
kubernetes_sd_configs	Kubernetes API (pods, services, endpoints)
consul_sd_configs	Consul service registry
dns_sd_configs	DNS SRV records
ec2_sd_configs	AWS EC2 instances
gce_sd_configs	GCP Compute Engine
file_sd_configs	JSON or YAML files (good for dynamic updates)

Kubernetes is the most common. Each pod gets discovered automatically and Prometheus reads its annotations to decide whether to scrape.

Recording rules

Precompute expensive queries on a schedule. Saves CPU and makes dashboards faster.

```
groups:
- name: http_rates
  interval: 30s
  rules:
    - record: job:http_requests:rate5m
      expr: sum by (job) (rate(http_requests_total[5m]))

    - record: job:http_errors:rate5m
      expr: sum by (job) (rate(http_requests_total{status=~"5.."}[5m]))
```

The new metric `job:http_requests:rate5m` shows up like a regular metric. Naming convention: `level:metric:operation` (level is what you've aggregated to, like `job` or `instance`).

Alerting rules

```
groups:
- name: http_alerts
  rules:
  - alert: HighErrorRate
    expr: |
      sum by (job) (rate(http_requests_total{status=~"5.."}[5m]))
      / sum by (job) (rate(http_requests_total[5m]))
      > 0.01
    for: 5m
    labels:
      severity: critical
      team: platform
    annotations:
      summary: "Error rate above 1% on {{ $labels.job }}"
      description: "{{ $value | humanizePercentage }}" of requests are failing"
```

- `expr` : the PromQL condition. While it returns rows, the alert is firing.
- `for` : how long the condition must hold before firing. Avoids flapping.
- `labels` : attached to the alert. Used by Alertmanager for routing.
- `annotations` : human-readable info. Templated.

Alert states:

- **Inactive**: expression returns nothing.
- **Pending**: expression returns rows, but not yet for the `for` duration.
- **Firing**: held for `for` duration, sent to Alertmanager.

Alertmanager

Receives alerts from Prometheus and decides what to do with them.


```

route:
  receiver: default
  group_by: [alertname, severity]
  group_wait: 30s
  group_interval: 5m
  repeat_interval: 4h
  routes:
    - match:
        severity: critical
      receiver: pagerduty
    - match:
        team: platform
      receiver: slack-platform

receivers:
  - name: default
    slack_configs:
      - api_url: https://hooks.slack.com/...
        channel: "#alerts"

  - name: pagerduty
    pagerduty_configs:
      - service_key: ...

  - name: slack-platform
    slack_configs:
      - api_url: ...
        channel: "#platform-alerts"

inhibit_rules:
  - source_match:
      severity: critical
    target_match:
      severity: warning
    equal: [alertname, job]

```

- **Grouping** bundles related alerts into one notification.
- **Routing** picks the right receiver based on labels.
- **Inhibition** suppresses lower-severity alerts when a higher-severity one fires (e.g., don't page on individual pod failures if the whole cluster is down).
- **Silences** mute alerts temporarily (planned maintenance).

Long-term storage

Prometheus stores around 15 days locally by default. For long-term retention and cross-cluster querying, layer on something else.

Tool	Notes
Thanos	Sidecars upload blocks to S3. Queriers federate across clusters. Mature.
Cortex	Multi-tenant. Designed to grow with many nodes. Now mostly evolved into Mimir.
Mimir (Grafana)	Cortex's successor. Multi-tenant, scales horizontally.
VictoriaMetrics	Faster ingest and lower memory than Prometheus. Long-term store or full replacement.

Pick one when you need years of metrics, multi-cluster federation, or downsampling for cost.

Java integration

The standard path in modern Spring Boot apps:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
<dependency>
  <groupId>io.micrometer</groupId>
  <artifactId>micrometer-registry-prometheus</artifactId>
</dependency>
```

```
management:
  endpoints:
    web:
      exposure:
        include: prometheus,health,info
```

Spring Boot now exposes `/actuator/prometheus` . Define custom metrics via Micrometer:

```

@Component
public class OrderMetrics {

    private final Counter ordersPlaced;
    private final Timer placeOrderLatency;

    public OrderMetrics(MeterRegistry registry) {
        this.ordersPlaced = Counter.builder("orders_placed_total")
            .description("Orders placed")
            .tag("service", "orders")
            .register(registry);
        this.placeOrderLatency = Timer.builder("orders_place_latency_seconds")
            .description("Latency to place an order")
            .register(registry);
    }
}

```

Micrometer translates the timer into a histogram on the Prometheus side. The bucket boundaries are configurable.

For non-Spring apps, use the official Prometheus Java client (`io.prometheus:simpleclient`) directly. Spring Boot apps should stick with Micrometer.

Grafana

The standard frontend for Prometheus. Build dashboards, run PromQL, set up alerts.

A typical setup:

- Prometheus as a Grafana data source (HTTP).
- Dashboards as JSON, checked into git.
- Variables (e.g., `$service` , `$instance`) for dropdown filters.

Grafana alerting can replace Prometheus alerting in some setups. Most teams keep Prometheus as the source of truth for alerts and use Grafana just for viewing.

Best practices

- **Pick label sets carefully.** Each unique label combination is a new time series. Costs grow fast.
- **Don't put per-request IDs as labels** (`user_id`, `request_id`, `trace_id`). Those belong in logs or traces.
- **Always wrap counters in `rate()` or `increase()`** . The raw counter resets on restart, so direct values don't tell you anything useful.
- **Pick the right window for `rate`** . Too short and the data is noisy. Too long and it lags. 5m is a good default.

- **Prefer histograms over summaries when aggregation matters.** Histograms can be combined across replicas. Summaries can't.
 - **Set up recording rules** for any PromQL expression you query often. Saves CPU.
 - **Name metrics with the conventions.** `_total` for counters, `_seconds` for durations, `_bytes` for sizes.
 - **Use `for:` in alerts.** Flapping alerts erode trust. Pending state catches noise.
 - **Set up Alertmanager inhibition** so high-severity alerts mute the low-severity ones.
 - **Cardinality budget your team.** A million series per service is rough at scale. Track it.
 - **Plan for long-term storage** if you need more than 2 weeks. Thanos, Mimir, or VictoriaMetrics.
-

Common gotchas

- **High cardinality blowup.** A label like `user_id` with a million values creates a million time series. Prometheus melts. Always cap your label cardinality.
 - **`rate()` on gauges returns garbage.** `rate` assumes a counter (monotonic increase). On a gauge, restarts and decrements break the math.
 - **`rate()` window shorter than scrape interval.** Need at least two samples. `rate(metric[15s])` with a 15s scrape interval often returns nothing.
 - **Counter resets on restart.** `rate()` handles a single reset gracefully but multiple resets in a window throw it off.
 - **Aggregating summaries.** You can't compute global p99 from per-instance p99 values. Use histograms.
 - **Forgetting `for:` on alerts.** Alerts that fire on a single 30s spike will page you constantly.
 - **Recording rule name collision.** Naming a recording rule the same as a scraped metric overwrites it. Use the `level:metric:operation` convention.
 - **Cross-cluster federation done wrong.** Federation is for selectively pulling aggregated metrics from one Prometheus to another. Pulling every series from every cluster doesn't scale. Use Thanos or Mimir.
 - **Alerts evaluating against missing data.** If `expr` returns no data, the alert won't fire. To alert on absence, use `absent()` or `absent_over_time()`.
 - **Time series with high churn.** Containers that come and go produce short-lived series. Each persists for the configured retention. Lots of churn means lots of series. Tune with staleness handling.
 - **Misusing Pushgateway.** Pushgateway exists for short-lived batch jobs that can't be scraped on a schedule. Real services should be scrapeable directly.
-

Quick reference

Concept	Key fact
Pull model	Prometheus scrapes targets, not the other way around
Counter	Monotonic. Use <code>rate()</code> or <code>increase()</code> .
Gauge	Up and down. Read directly.
Histogram	Bucketed observations. Use <code>histogram_quantile</code> .
Summary	Pre-computed quantiles. Hard to aggregate. Prefer histograms.
<code>rate(metric[5m])</code>	Per-second rate over the last 5 minutes
<code>increase(metric[1h])</code>	Total increase over the last hour
<code>histogram_quantile(0.95, ...)</code>	p95 from a histogram
<code>sum by (label) (...)</code>	Group by a label
Recording rule	Precomputed query, named <code>level:metric:operation</code>
Alerting rule	<code>expr + for + labels + annotations</code>
Alertmanager	Routes, groups, inhibits, silences alerts
Exporters	<code>node_exporter</code> , <code>mysqld_exporter</code> , <code>redis_exporter</code> , etc.
Micrometer	The Java metrics facade. Default in Spring Boot.
Cardinality	Each unique label combo is a new time series. Bound it.
Thanos / Mimir	Long-term storage and cross-cluster federation